

# LAPORAN PRAKTIKUM BIG DATA

## Praktikum 2



OLEH :

JARDEL POLIVIERA

2411533012

DOSEN PENGAMPU:

AFDHAL DINILHAK. M.Kom

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG 2026

Link repo :

[https://github.com/JardelPoliviera/BigData26\\_B\\_2411533012\\_JardelPoliviera](https://github.com/JardelPoliviera/BigData26_B_2411533012_JardelPoliviera)

Link GDrive :

[https://drive.google.com/drive/folders/1Gjtd2beuucuf3Hwnllp6nuvtqGo\\_cz\\_O?usp=drive\\_link](https://drive.google.com/drive/folders/1Gjtd2beuucuf3Hwnllp6nuvtqGo_cz_O?usp=drive_link)

## A. LANGKAH PRAKTIKUM

### 1. K-1. Import Library dan Inisialisasi

```
import numpy as np
import pandas as pd
from faker import Faker
import random
```

Langkah pertama adalah mengimpor pustaka yang dibutuhkan. Pustaka numpy digunakan untuk operasi komputasi numerik angka. Pustaka pandas digunakan untuk memanipulasi data tabular. Pustaka Faker dipakai untuk membangkitkan data sintesis acak. Pustaka random digunakan untuk menjalankan fungsi acak bawaan Python.

### 2. K-2. Membuat Dataset Sintetis

```
SEED = 42
np.random.seed(SEED)
random.seed(SEED)
faker = Faker('id-ID')
faker.seed(SEED)

N = 500
kategori_produk = ["Elektronik", "Fashion", "Kesehatan", "Rumah Tangga", "Olahraga", "Buku"]
metode_bayar = ["Transfer Bank", "E-Wallet", "COD", "Kartu Kredit"]

rows = []
for i in range(1, N + 1):
    trx_id = f"TRX{i:05d}"
    nama_pelanggan = faker.name()
    produk = faker.word().capitalize() + " " + random.choice(["Pro", "Lite", "Max", "Basic", ""])
    kategori = random.choice(kategori_produk)
    harga_desar = random.choice([15000, 25000, 50000, 75000, 120000, 200000, 500000, 1200000])
    qty = random.randint(1, 5)

    # Variasi format harga: angka ribuan, ada "Rp", ada desimal ".", ada spasi
    harga_variants = [
        str(harga_desar),
        f"Rp{harga_desar:,.1f}.replace('.', ',')",
        f"Rp{harga_desar:,.1f}",
        f"{harga_desar:,.1f}"
    ]
    harga = random.choice(harga_variants)

    # Variasi format tanggal: DD-MM/YYYY, DD-MM-YYYY
    tgl = faker.date_between(start_date='-50d', end_date='today')
    tgl_variants = [tgl.strftime("%d-%m-%d"), tgl.strftime("%d/%m/%Y"), tgl.strftime("%d-%m-%Y")]
    tanggal = random.choice(tgl_variants)

    metode = random.choice(metode_bayar)
    if random.random() < 0.3:
        metode = metode.lower()
    if random.random() < 0.2:
        kategori = kategori.upper() + " "

    kota = faker.city()
    rating = random.choice([1, 2, 3, 4, 5, None, None]) # rating optional

    rows.append({
        "transaction_id": trx_id,
        "customer_name": nama_pelanggan,
        "product_name": produk.strip(),
        "category": kategori,
        "price": harga,
        "quantity": qty,
        "payment_method": metode,
        "transaction_date": tanggal,
        "shipping_city": kota,
        "rating": rating,
    })

df = pd.DataFrame(rows)

# Sertikan missing value pada beberapa kolom
for col, frac in [{"customer_name": 0.02}, {"shipping_city": 0.03}, {"payment_method": 0.015}]:
    idx = df.sample(frac=frac, random_state=SEED).index
    df.loc[idx, col] = np.nan

# Duplikasi 15 baris (maksimalisasi transaksi yang tercatat dua kali)
dup_rows = df.sample(n=15, random_state=SEED)
df = pd.concat([df, dup_rows], ignore_index=True)
df = df.sample(frac=1, random_state=SEED).reset_index(drop=True)

df.to_csv("transaksi_montah.csv", index=False)
print(f"Jumlah baris: ", len(df))

Jumlah baris: 515
```

Tahap ini mensimulasikan proses pengumpulan data mentah. Kita menetapkan nilai awal acak bernilai 42 agar hasil pembuatan data selalu konsisten saat dijalankan ulang. Proses dilanjutkan dengan membuat 500 baris data transaksi fiktif lengkap dengan variasi penulisan harga dan tanggal. Setelah data terbentuk, program sengaja menyuntikkan nilai kosong pada beberapa kolom dan menggandakan 15 baris data untuk meniru masalah data mentah di dunia nyata.

### 3. K-3. Deteksi dan Penanganan Missing Value

```
# Cek missing value
print(df.isnull().sum())

# Penanganan missing value
df = df.dropna(subset=["customer_name", "payment_method"])
df["shipping_city"] = df["shipping_city"].fillna("Tidak Diketahui")
print("Jumlah baris setelah penanganan missing value:", len(df))
```

  

```
... transaction_id      0
   customer_name      20
   product_name       0
   category           0
   price              0
   quantity           0
   payment_method     16
   transaction_date    0
   shipping_city      30
   rating             166
   dtype: int64
Jumlah baris setelah penanganan missing value: 495
```

Langkah ini berfungsi mendeteksi dan menangani data yang hilang. Kita menghapus keseluruhan baris transaksi yang tidak memiliki nama pelanggan dan metode pembayaran karena dua informasi tersebut berstatus wajib. Untuk kolom kota pengiriman yang bernilai kosong, kita menambalnya dengan teks Tidak Diketahui agar baris data berharga tersebut tetap bisa digunakan. Kolom rating dibiarkan kosong karena pemberian rating bersifat murni opsional bagi pelanggan.

### 4. K-4. Deteksi dan Penanganan Duplicate

```
print("Baris duplicate (semua kolom sama):", df.duplicated().sum())
print("transaction_id duplicate:", df['transaction_id'].duplicated().sum())

df = df.drop_duplicates()
print("Jumlah baris setelah drop_duplicates():", len(df))
```

  

```
Baris duplicate (semua kolom sama): 5
transaction_id duplicate: 5
Jumlah baris setelah drop_duplicates(): 490
```

Proses ini bertujuan mencari dan menghapus baris data yang terindikasi persis sama. Penghapusan duplikat ini sangat penting untuk memastikan tidak ada satupun transaksi yang tercatat lebih dari sekali di dalam sistem pencatatan akhir.

## 5. K-5. Koreksi Tipe Data dan Standardisasi Format

```
# a. Standardisasi teks kategorikal
for col in ["category", "payment_method", "shipping_city"]:
    df[col] = df[col].astype("string").str.strip().str.title()

# "Cod" adalah singkatan; kembalikan ke huruf kapital penuh
df["payment_method"] = df["payment_method"].replace({"Cod": "COD"})

# b. Koreksi tipe data pada kolom price
def bersihkan_harga(x):
    if pd.isna(x):
        return np.nan
    x = str(x).strip().replace("Rp", "").replace(".", "").replace(",", ".")
    try:
        return float(x)
    except ValueError:
        return np.nan

df["price"] = df["price"].apply(bersihkan_harga)

# c. Standardisasi format tanggal ke YYYY-MM-DD
def parse_tanggal(x):
    for fmt in ("%Y-%m-%d", "%d/%m/%Y", "%d-%m-%Y"):
        try:
            return pd.to_datetime(x, format=fmt)
        except ValueError:
            continue
    return pd.NaT

df["transaction_date"] = df["transaction_date"].apply(parse_tanggal).dt.strftime("%Y-%m-%d")

# d. Finalisasi tipe data
df["quantity"] = df["quantity"].astype(int)
df["price"] = df["price"].astype(float)
```

Tahap ini menyeragamkan format teks serta memperbaiki tipe data yang salah. Teks pada kategori produk, metode bayar, dan kota diubah agar memiliki huruf kapital di setiap awal kata. Khusus metode bayar di tempat, teks dikembalikan menjadi huruf kapital penuh. Kolom harga dibersihkan dari simbol mata uang beserta tanda baca, kemudian diubah menjadi tipe data angka desimal. Kolom tanggal juga ikut diseragamkan menjadi wujud baku tahun bulan dan hari.

## 6. K-6. Ekspor Dataset Bersih

```
df.to_csv("transaksi_bersih.csv", index=False)
print("Dataset bersih tersimpan:", len(df), "baris")

... Dataset bersih tersimpan: 490 baris
```

Data transaksi yang sudah melewati seluruh proses pembersihan di atas kemudian diekspor ke dalam bentuk file CSV baru. File akhir ini bersih dari kesalahan, berjumlah 490 baris.

## B. ANALISIS HASIL

Berdasarkan proses pembersihan data mentah di atas, sebanyak 25 baris data telah dibuang dari total awal 515 baris data mentah. Rincian penghapusan ini meliputi 15 baris akibat duplikasi sistem dan 10 baris akibat ketiadaan informasi wajib dari pembeli atau metode pembayaran. Kehilangan sebagian kecil data dalam tahap ini merupakan sebuah kondisi wajar yang membuktikan bahwa alur pembersihan bekerja dengan semestinya. Kolom rating sengaja dibiarkan memiliki nilai kosong guna mencegah distorsi realitas apabila harus diisi dengan angka tebakan asal.

## C. Studi Kasus

1. Penyebab perbedaan angka laporan: Perbedaan angka terjadi karena tim teknologi informasi merujuk pada jumlah keseluruhan data mentah yang baru masuk, sedangkan tim keuangan merujuk pada data yang sudah melewati tahap pembersihan. Angka menyusut karena ada pembuangan 15 baris transaksi ganda dan 10 baris transaksi cacat tanpa identitas lengkap.
2. Kaitan 490 baris dengan konsep Veracity: Laporan berisi 490 baris data terbukti jauh lebih benar. Dalam dimensi lima V, istilah Veracity merujuk pada tingkat keakuratan sebuah data. Data mentah awal memiliki tingkat keakuratan rendah akibat anomali di dalamnya, sedangkan 490 baris data sisa memiliki tingkat keakuratan tinggi sehingga sangat terpercaya untuk dasar pengambilan keputusan keuangan.
3. Penjelasan nilai kosong pada rating: Memberikan angka tebakan pada kolom ulasan akan memanipulasi fakta. Penjelasan paling logis bagi tim keuangan adalah pencarian nilai rata-rata ulasan harus dilakukan murni dari pembagian total nilai ulasan terhadap jumlah transaksi spesifik yang benar-benar memberikan penilaian akhir saja.

## D. LATIHAN

1. Mengubah SEED menjadi 7

```
SEED = 7
np.random.seed(SEED)
random.seed(SEED)
fake = Faker("id_ID")
Faker.seed(SEED)
```

Saat nilai awal acak diubah dari angka 42 menjadi 7, jumlah baris data mentah tetap berwujud 515 baris dan jumlah baris data bersih tetap berwujud 490 baris. Kesamaan nominal akhir ini terjadi karena porsi penyuntikan data hilang dan penggandaan duplikat sudah diatur secara ketat menggunakan metode persentase pecahan di dalam skrip, sehingga jumlah data yang dihapus selalu bernilai konsisten meski isi datanya berbeda.

2. Menambahkan kolom is\_valid\_price

```
df["is_valid_price"] = df["price"] > 0
df.to_csv("transaksi_bersih.csv", index=False)
print("Dataset bersih tersimpan:", len(df), "baris")

... Dataset bersih tersimpan: 490 baris
```

Kita menyisipkan satu kolom baru untuk memeriksa kecacatan harga. Kolom ini bekerja dengan cara mengevaluasi apakah nominal di dalam kolom harga bernilai lebih besar dari angka nol, lalu menyimpan status tersebut dalam bentuk pernyataan benar atau salah.

3. Menghitung transaksi per kategori

```
kategori_counts = df["category"].value_counts()

print("Persebaran Jumlah Transaksi per Kategori:")
print(kategori_counts)

Persebaran Jumlah Transaksi per Kategori:
category
Rumah Tangga    91
Kesehatan       86
Buku            81
Fashion         80
Elektronik      78
Olahraga        74
Name: count, dtype: Int64
```

Pencarian jumlah ini memanfaatkan fungsi hitung nilai untuk melihat distribusi pembelian. Hasil eksekusi menunjukkan Kategori Rumah Tangga menempati urutan puncak dengan total 91 transaksi, diikuti Kesehatan 86 transaksi, Buku 81 transaksi, Fashion 80 transaksi, Elektronik 78 transaksi, dan ditutup oleh Olahraga dengan total 74 transaksi.

## E. PERTANYAAN EVALUASI

1. Perbedaan dropna dan fillna: Fungsi pertama berguna untuk menghanguskan baris secara menyeluruh jika ditemukan sebuah sel kosong, dan paling cocok disematkan pada kolom berisi data identitas mutlak. Sementara itu fungsi kedua berguna untuk mempertahankan eksistensi baris dengan cara menyematkan teks buatan pada sel kosong, dan paling cocok dipakai pada kolom informasi pelengkap.
2. Alasan drop\_duplicates di tahap awal: Penghapusan duplikat pada urutan pertama sangat menghemat sumber daya waktu serta memori komputasi mesin. Sistem tidak perlu membuang energi untuk memperbaiki tipe data rumit pada baris kembar yang pada akhirnya tetap akan dibuang.
3. Risiko parameter mixed dan dayfirst True: Kombinasi perintah tanggal campuran dan pemaksaan pembacaan format hari pada urutan depan berisiko fatal merusak wujud data awal. Tanggal yang sejatinya sudah tertulis rapi dengan format baku tahun bulan hari bisa ikut terbalik posisinya secara keliru dan melahirkan ambiguitas data.
4. Alasan perbedaan nasib kolom rating dan shipping\_city: Ulasan dibiarkan kosong karena mencerminkan sifat sukarela pelanggan, sehingga mencegah munculnya data fiktif. Sementara itu kota pengiriman ditambal menggunakan kalimat Tidak Diketahui demi menyelamatkan catatan pembelian sang pelanggan agar nominal uangnya tetap bisa direkap ke dalam laporan laba bulanan perusahaan.
5. Kaitan praktikum dengan Veracity: Rangkaian praktikum pembersihan ini menjadi implementasi konkret dari penguatan konsep Veracity. Kita membongkar sekumpulan data kotor agar tingkat akurasi serta tingkat kepercayaannya meningkat tajam, guna mencegah terjadinya kecacatan rumusan saat data tersebut disalurkan menuju tahapan analisis akhir.